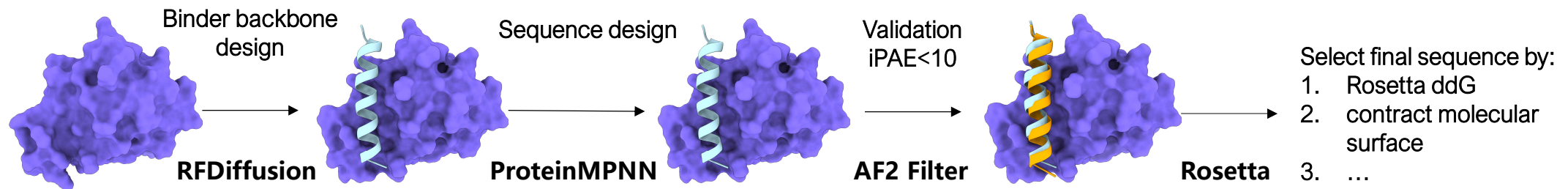


Protein Design via Deep Learning

Naf Guo
2026

Protein Design Workflow

Example pipeline of AI based protein binder design



- RFDiffusion was used to design backbone based on target protein;
- ProteinMPNN was used to predict sequences that could fold into designed backbone;
- AF2 was used to filter bad sequences (**most important part in the whole pipeline**);
- ddG & CMS were used to further select sequences for experiment testing.

bioRxiv. 2024, 2024-09.

bioRxiv. 2024, 2024-11.

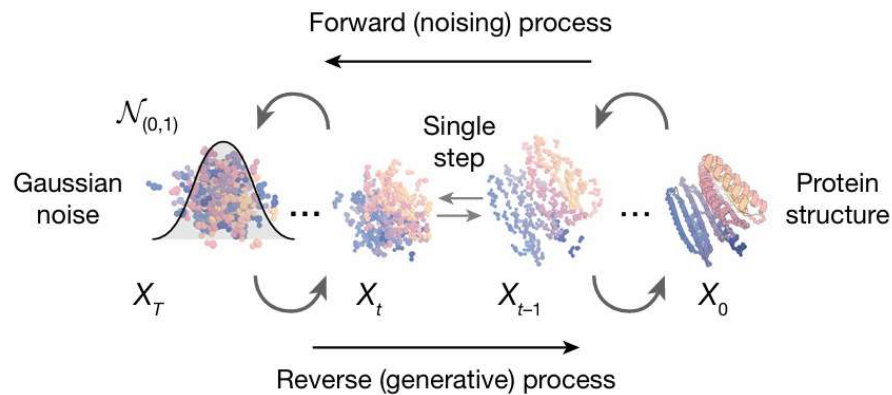
Science. 2022, 378, 49-56.

Nature. 2023, 620, 1089-1100.

Nature Communications. 2023, 14, 2625.

RFDiffusion

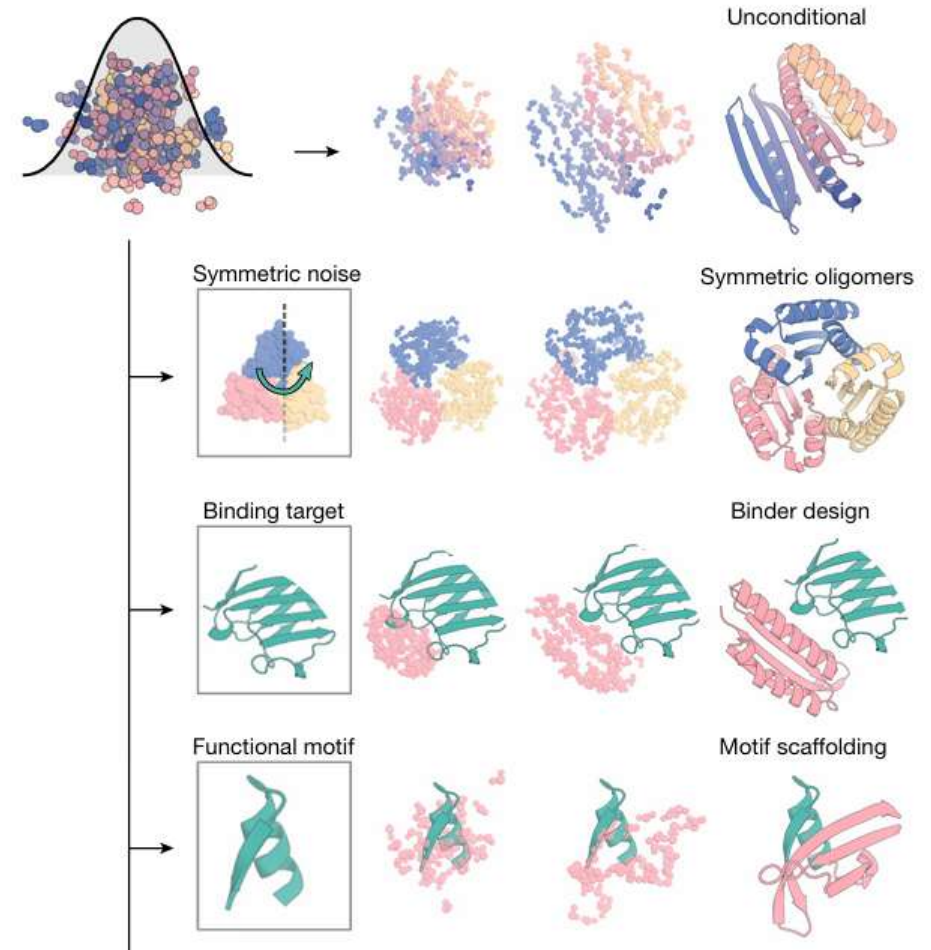
Diffusion model



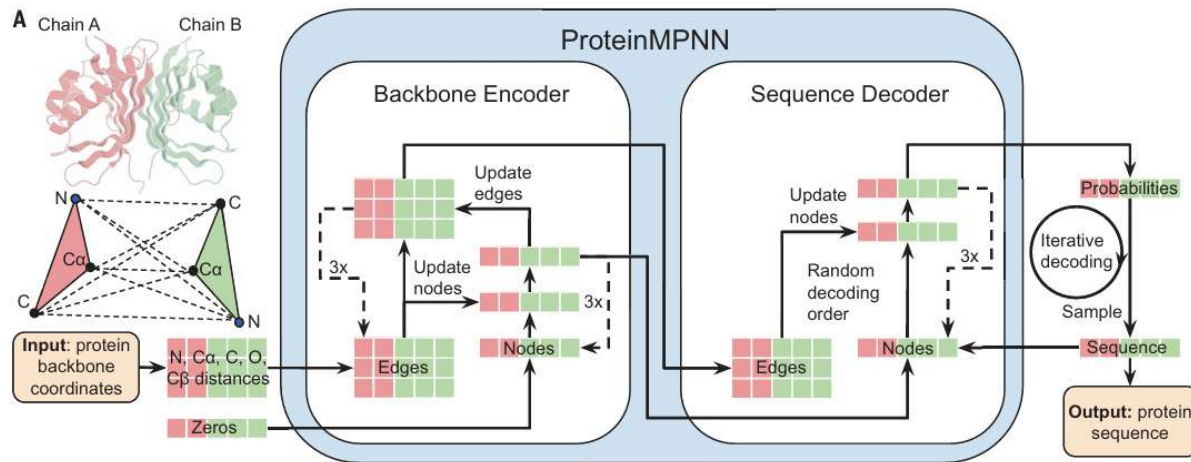
A generative model like diffusion model would learn data distribution from training data, then user can sample new data point from that distribution.

In diffusion model, we need a network that can predict “noise” of the same shape with input data, here is a protein structure, RosettaFold/AF2 or any structure prediction network would do the trick.

Special cases like binder design is just conditional sampling.



ProteinMPNN



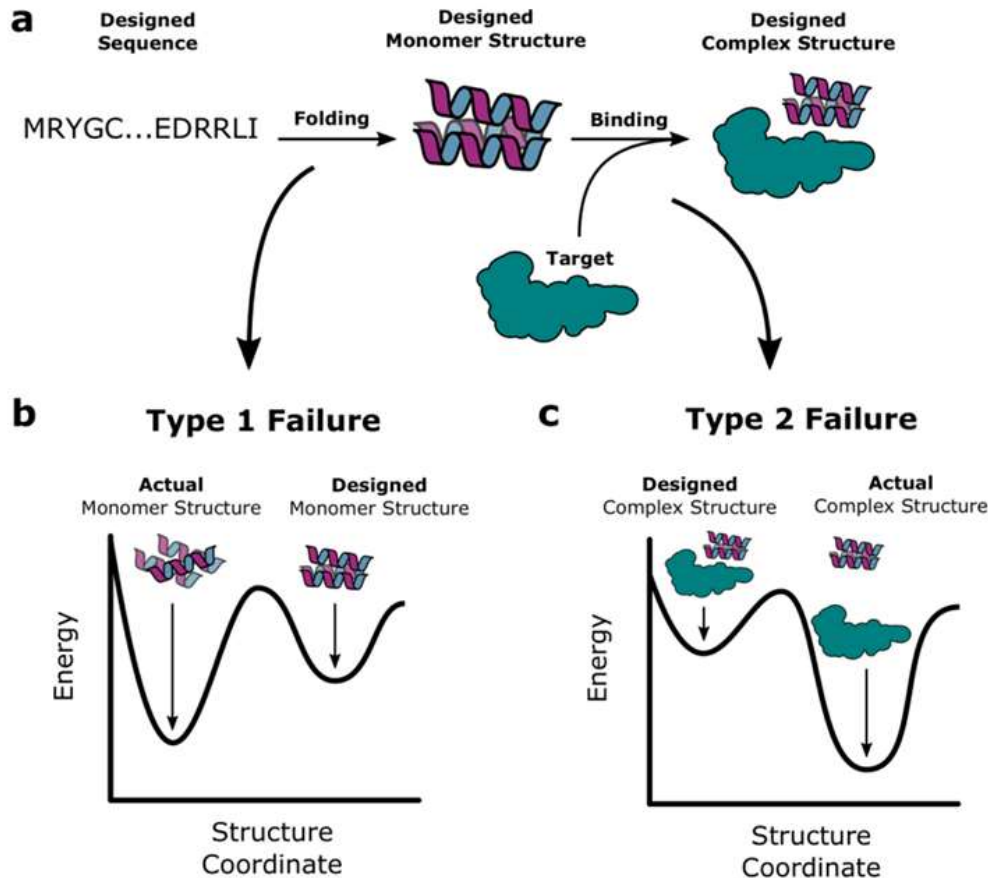
Given a set of protein backbone coordinates, ProteinMPNN can predict sequences might be able to fold into that scaffold.

Recover rate in PDB test is 50.8%

Never mind, most bad sequences would be filtered out with AF2 filter.

Noise level when training: 0.00 Å/0.02 Å	Modification	Number of parameters in millions	PDB test accuracy (%)	PDB test perplexity	AlphaFold model accuracy (%)
Baseline model	None	1.381	41.2/40.1	6.51/6.77	41.4/41.4
Experiment 1	Add N, Cα, C, Cβ, O distances	1.430	49.0/46.1	5.03/5.54	45.7/47.4
Experiment 2	Update encoder edges	1.629	43.1/42.0	6.12/6.37	43.3/43.0
Experiment 3	Combine 1 and 2	1.678	50.5/47.3	4.82/5.36	46.3/47.9
Experiment 4	Experiment 3 with random decoding	1.678	50.8/47.9	4.74/5.25	46.9/48.5

AF2 Filter



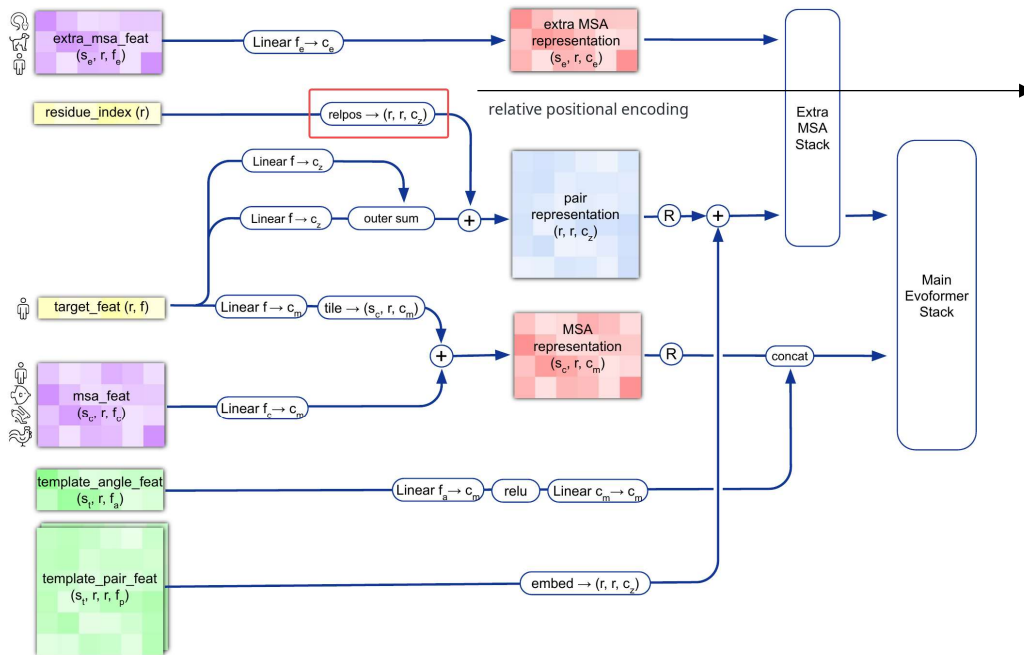
2 type of failures in binder design

- Designed sequences failed to fold into designed structure.
- Designed sequences failed to bind with target protein.

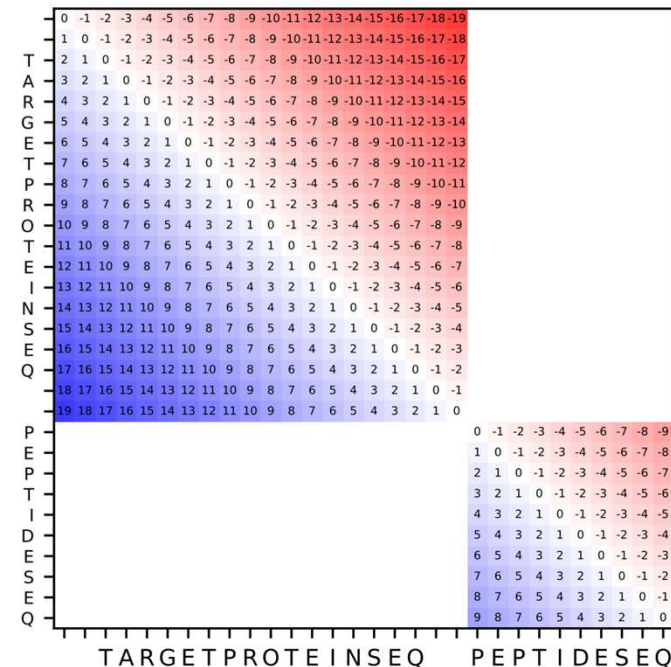
AF2 filter

- Monomer structure prediction: if predicted structure of designed sequence similar with designed structures;
- Complex structure prediction: if predicted complex structure agree with designed complex structure;
- This is the secret of high hit rate.

Cyclic Peptide Structure Prediction

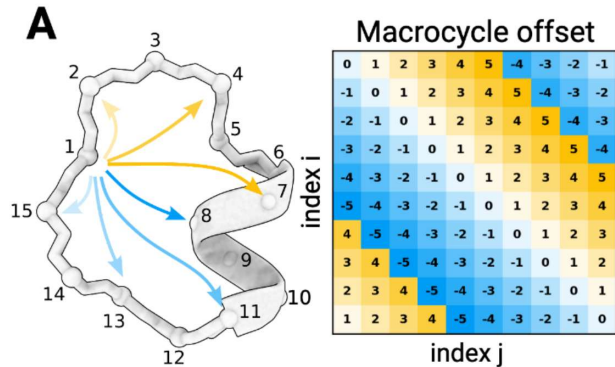


Transformer's backbone network cannot utilize sequence order information. So, a relative positional encoding is needed to tell the transformer the relative position of different residues.

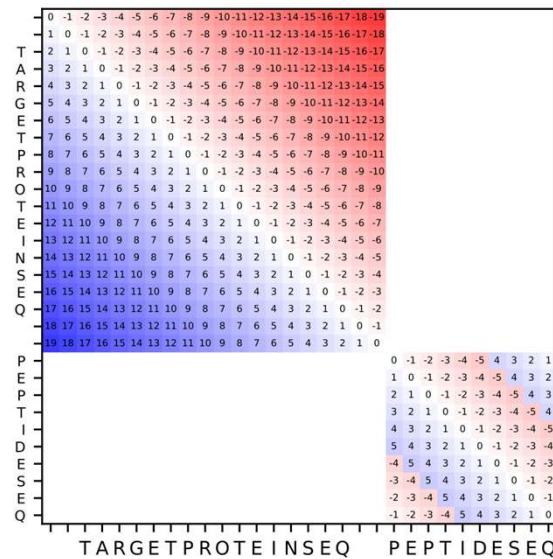


Supplementary Figure 1 | Input feature embeddings. Dimension names: r : residues, f : features, c : channels, s_c : clustered MSA sequences, s_e : extra MSA sequences, s_t : template sequences. Operator relpos is defined in Algorithm 4. Template embedding and extra MSA stack are explained in subsection 1.7. Recycling injection is denoted with R, see subsection 1.10 for details.

Cyclic Peptide Structure Prediction

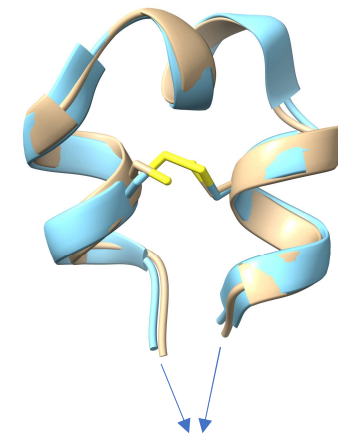
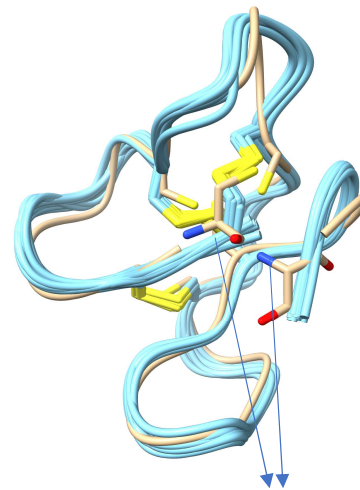


Changing the relative position encoding to a cyclic one would make AF2 or any AF2 like model can predict macrocycle structures.



2mw0

5kx1



The two ends are not connected due to display issues, but the distance between them is that of a chemical bond.

Cyclic Peptide Backbone Generation

Modified 3 functions in RFDiffusion could make it can generate macrocycle scaffold

<https://github.com/RosettaCommons/RFDiffusion/blob/main/rfdiffusion/Embeddings.py>

https://github.com/RosettaCommons/RFDiffusion/blob/main/rfdiffusion/Util_module.py

```
class PositionalEncoding2D(nn.Module):
    # Add relative positional encoding to pair features
    def __init__(self, d_model, minpos=32, maxpos=32, p_drop=0.1):
        super(PositionalEncoding2D, self).__init__()
        self.minpos = minpos
        self.maxpos = maxpos
        self.nbin = abs(minpos)+maxpos+1
        self.emb = nn.Embedding(self.nbin, d_model)
        self.drop = nn.Dropout(p_drop)

    def forward(self, x, idx):
        bins = torch.arange(self.minpos, self.maxpos, device=x.device)
        seqsep = idx[:,None,:] - idx[:,None,:] # (B, L, L)
        #
        ib = torch.bucketize(seqsep, bins).long() # (B, L, L)
        emb = self.emb(ib) # (B, L, L, d_model)
        x = x + emb # add relative positional encoding
        return self.drop(x)
```

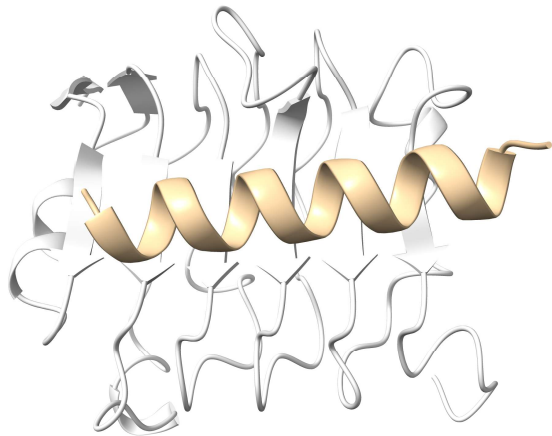
```
def make_full_graph(xyz, pair, idx, top_k=64, kmin=0):
    ...
    Input:
    - xyz: current backbone coordinates (B, L, 3, 3)
    - pair: pair features from Trunk (B, L, L, E)
    - idx: residue index from ground truth pdb
    Output:
    - G: defined graph
    ...
    B, L = xyz.shape[:2]
    device = xyz.device

    # seq sep
    seqsep = idx[:,None,:] - idx[:,None,:]
    b,i,j = torch.where(seqsep.abs() > 0)

    src = b*L+i
    tgt = b*L+j
    G = dgl.graph((src, tgt), num_nodes=B*L).to(device)
    G.edata['rel_pos'] = (xyz[b,i,:]-xyz[b,j,:]).detach() # no gradient through basis function
    return G, pair[b,i,j][...],None]
```

```
def make_topk_graph(xyz, pair, idx, top_k=64, kmin=32, eps=1e-6):
    ...
    Input:
    - xyz: current backbone coordinates (B, L, 3, 3)
    - pair: pair features from Trunk (B, L, L, E)
    - idx: residue index from ground truth pdb
    Output:
    - G: defined graph
    ...
    B, L = xyz.shape[:2]
    device = xyz.device

    # distance map from current CA coordinates
    D = torch.cdist(xyz, xyz) + torch.eye(L, device=device).unsqueeze(0)*999.9 # (B, L, L)
    # seq sep
    seqsep = idx[:,None,:] - idx[:,None,:]
    sep = seqsep.abs() + torch.eye(L, device=device).unsqueeze(0)*999.9
    D = D + sep*eps
```



RFDiffusion output with normal positional encoding



RFDiffusion output with cyclic positional encoding